AI-Driven Predictive & Preventive Maintenance for a 250-Node NS-2 Network

In-Depth File-by-File Explanation and Viva Study Guide

Core flow: NS-2 Simulation → Data Collection → Data Processing → ML Dataset → Random Forest → Failure Prediction → Risk Classification → Preventive Decision → Graphs

1. Big Picture

The project has five layers: network simulation, data collection/analysis, ML dataset preparation, AI prediction, and preventive maintenance. NS-2 creates the network experiment. Python processes the simulation output, trains the ML model, predicts risk, generates graphs, and demonstrates the preventive decision.

Architecture

simulation250.tcl → energy250.csv + network250.tr → Python analysis → ml_dataset.csv → Random Forest → failure_prediction_model.pkl → node predictions → integrated results → graphs + preventive decision.

2. simulation250.tcl

This is the main NS-2 experiment file. It creates the 250-node wireless/mesh network, configures routing, traffic, energy behaviour, simulation duration and output recording. The project uses AODV routing (Ad hoc On-Demand Distance Vector) and CBR traffic (Constant Bit Rate). AODV can discover routes on demand; CBR creates ongoing packet traffic so nodes actually consume energy. Main outputs are results/energy250.csv and results/network250.tr.

3. test_energy.tcl

This is a testing/debugging NS-2 script used to verify the energy model. The key check is that nodes start with an energy value and their remaining energy decreases as communication occurs.

4. energy250.csv

This is the main energy output. Energy is sampled approximately every 5 seconds for 250 nodes. With 50 time points there are 12,500 observations plus the header. It answers: how much energy does each node have at a given time? Energy is one of the main health indicators used by the predictive-maintenance system.

5. network250.tr

This is the raw NS-2 trace file. It records packet-level network events such as transmission, reception, forwarding, dropping and timing. Think of it as the detailed event log of the simulated network.

6. node_network_analysis.py

This Python script reads network250.tr and extracts useful network statistics such as packets sent, packets received, packets lost, packet loss and delay. The project analysis reported 15,712 packets sent, 5,059 received and 10,653 lost. It produces node_network_analysis.csv.

7. node_network_analysis.csv

This is the processed network-performance dataset. It is easier to inspect and combine than the raw trace. It represents information extracted from network250.tr rather than being a separate simulation.

8. create_ml_dataset.py

This is the bridge from simulation to machine learning. It prepares features such as current energy, energy decay and energy consumed. It also creates the failure_risk target using future-energy behaviour and a project-specific critical-energy threshold. The final project used CRITICAL_ENERGY = 45. This is an experimental project threshold, not a universal wireless-network failure standard. The final dataset had 11,750 samples: 10,922 healthy/low-risk labels and 828 risk labels.

9. ml_dataset.csv

This is the dataset used to train the Random Forest. Conceptually each row contains Energy, Energy decay, Energy consumed and Failure risk. The first three are input features and failure_risk is the target.

10. train_model.py

This is the main ML training script. It trains a Random Forest Classifier using Energy, Energy decay and Energy consumed to predict failure_risk. The data was split into 9,400 training samples and 2,350 testing samples. Reported test accuracy was 99.83%. The confusion matrix was [[2182,2],[2,164]]. The model also used balanced class weighting.

11. Random Forest concept

Random Forest combines many decision trees. One tree may learn rules involving energy or energy decay; many trees vote together for the final class. This is an ensemble classification method. In this project, model feature importance was approximately Energy 73.6%, Energy decay 13.7%, Energy consumed 12.7%. These values describe this trained model and dataset, not universal physical laws.

12. failure_prediction_model.pkl

This file stores the trained Random Forest using Python Pickle. Saving the model means it can be loaded later for prediction without retraining every time. Flow: ml_dataset.csv → train_model.py → trained model → failure_prediction_model.pkl.

13. predict_nodes.py

This script loads the trained model and predicts node risk. It uses the node features and produces prediction information such as failure probability and risk level. The final distribution was HIGH 205, MEDIUM 1 and LOW 44, totaling 250 nodes. High-risk does not mean a node has already failed; it means the model classified it as high-risk under the project setup.

14. final_node_predictions.csv

This stores the node-level AI predictions. It can contain node ID, energy features, failure probability and risk level. It answers: which nodes need attention from the preventive-maintenance layer?

15. integrate_results.py

This combines different project outputs, such as energy information, network metrics and AI predictions, into one unified result. It prevents the user from having to inspect multiple files separately.

16. final_integrated_results.csv

This is the combined result table. It includes information such as node ID, energy, energy decay, energy consumed, failure probability, risk level, packet loss, delay and preventive action.

17. generate_graphs.py

This script generates most of the project graphs: average delay, energy decay, energy levels, energy versus failure, energy versus time, failure probability, network topology, packet loss and risk distribution.

18. energy_time_graph.py

This script specifically visualizes average energy over time. The project showed approximately 100 initial average energy and 42.54 final average energy, demonstrating a decreasing energy trend during simulation.

19. network_topology.py

This visualizes the 250-node network topology. It helps explain network scale, node arrangement and the distribution of nodes or risk categories.

20. preventive_simulation.py

This demonstrates the preventive-maintenance decision. A high-risk node is identified, an alternate lower-risk node is selected, and traffic rerouting is demonstrated conceptually. Before: SOURCE → HIGH-RISK NODE → DESTINATION. Preventive: SOURCE → ALTERNATE LOW-RISK NODE → DESTINATION. Important limitation: the current script demonstrates the decision layer; it does not prove that the original NS-2 trace was dynamically rerouted and rerun. A second NS-2 experiment with feedback from the prediction would be the next extension.

21. Understand the Graphs

average_delay.png: average time taken by packets to reach destinations.

energy_decay.png: how quickly node energy is changing.

energy_levels.png: remaining energy of nodes.

energy_vs_failure.png: relationship between energy behaviour and predicted failure risk.

energy_vs_time.png: how average energy changes as simulation time increases.

failure_probability.png: AI-predicted failure probability/risk.

network_topology.png: structure of the 250-node network.

packet_loss.png: packets lost during communication.

risk_distribution.png: number of Low, Medium and High risk nodes.

22. Raw Data → Processed Data → ML → Prediction

Level 1: network250.tr is raw packet-level simulation data. Level 2: node_network_analysis.csv contains processed network statistics. Level 3: ml_dataset.csv contains ML features and the target. Level 4: failure_prediction_model.pkl contains the trained model. Level 5: final_node_predictions.csv contains AI predictions. Level 6: final_integrated_results.csv combines the main information.

23. Why NS-2 and Python?

NS-2: network nodes, routing, packets, wireless communication and energy simulation.
Python: CSV processing, statistics, machine learning, prediction, visualization and preventive decision logic.

24. Input → Process → Output

simulation250.tcl → NS-2 configuration → 250-node simulation → energy250.csv + network250.tr

node_network_analysis.py → network250.tr → packet/delay/loss analysis → node_network_analysis.csv

create_ml_dataset.py → energy data → feature engineering + risk labels → ml_dataset.csv

train_model.py → ML dataset → Random Forest training → failure_prediction_model.pkl

predict_nodes.py → model + node data → risk prediction → final_node_predictions.csv

integrate_results.py → multiple results → combined table → final_integrated_results.csv

generate_graphs.py → results → visualization → PNG graphs

preventive_simulation.py → predictions → preventive decision demonstration → rerouting output

25. Important Viva Questions

Q: What is the objective?
A: Predict possible node failures in a 250-node wireless/mesh network using ML and take preventive action before the predicted failure condition.

Q: Why NS-2?
A: It provides a network simulation environment for packet-level, routing and energy behaviour.

Q: Which ML algorithm?
A: Random Forest Classifier.

Q: Which features?
A: Energy, energy decay and energy consumed.

Q: What is the target?
A: failure_risk.

Q: What is the .pkl file?
A: The saved trained Random Forest model.

Q: Does 205 high-risk mean 205 nodes failed?
A: No. It means 205 simulated nodes were classified as high-risk by the model under the project setup.

Q: What does 99.83% mean?
A: It is the reported test accuracy on the generated simulation dataset, not guaranteed real-world accuracy.

Q: Was the original NS-2 simulation dynamically rerouted using AI?
A: The current implementation demonstrates the preventive decision/rerouting layer. A second NS-2 run with prediction feedback would be the next extension for direct simulation validation.

26. One-Minute Explanation

Our project is an AI-driven predictive and preventive maintenance system for a 250-node wireless network. First, NS-2 simulates 250 nodes with AODV routing, CBR traffic and an energy model. The simulation generates energy data and a packet-level trace. Python processes these outputs and creates an ML dataset using node energy, energy decay and energy consumption as features. A future-energy-based rule provides the failure-risk target. We train a Random Forest classifier and save the model. The trained model is then used to predict node failure risk and classify nodes as Low, Medium or High risk. Finally, the preventive layer demonstrates identifying a high-risk node and selecting an alternate lower-risk node for rerouting. Graphs visualize energy, delay, packet loss, failure probability, topology and risk distribution.

27. The Key Sentence

Simulation $\rightarrow$ Dataset $\rightarrow$ Prediction $\rightarrow$ Prevention.

If you understand what information moves between these four stages, you understand the central idea of the project.